

PhoneOS — Complete System Setup, Replication, Development & Deployment Manual

Subtitle: DroneOS + PX4/Pixhawk + MAVSDK + Relay + Mobile Ground Control Application

Project Version: 1.0 (2026-08-24)

Target Hardware: Raspberry Pi (Drone 1, 2, 3), Pixhawk / PX4 Autopilots, PC/Laptop/Android (Ground Station)

Target Software: Python 3.12+, MAVSDK, Vite + React + Capacitor (Mobile), Linux

Document Purpose

This document serves as the **REPLICATION + INSTALLATION + DEVELOPMENT + OPERATIONS MANUAL**. A completely new developer must be able to use this documentation to clone the PhoneOS project, prepare the required Raspberry Pi systems, install all dependencies, configure the drones, build/run the mobile app, start DroneOS and Relay, test communication, troubleshoot common errors, and reproduce the complete working system.

IMPORTANT DOCUMENTATION RULE

Throughout this manual, commands are labeled based on where they should be executed: - [PC/LAPTOP] - Executed on your development machine or Ground Station. - [DRONE 1 - Raspberry Pi] - Executed on the Raspberry Pi for Drone 1. - [DRONE 2 - Raspberry Pi] - Executed on the Raspberry Pi for Drone 2. - [DRONE 3 - Raspberry Pi] - Executed on the Raspberry Pi for Drone 3.

1. PHONEOS ARCHITECTURE

The PhoneOS architecture bridges a Mobile App Ground Control Station to multiple drones running Pixhawk hardware.

```
Mobile App | | WebSocket / Network v Relay | | UDP v DroneOS | v Flight
Manager / Command Handler / Safety | v MAVSDK | v MAVSDK Server | | UART v
```

Multi-Drone Operation

Each drone in the swarm (Drone 1, Drone 2, Drone 3) runs its own dedicated Raspberry Pi with its own instance of DroneOS, connecting to its own Pixhawk via UART.

- **Drone 1:** Runs `DroneOS/main.py + relay/relay.py`. Acts as the primary relay hub and runs DroneOS.
- **Drone 2:** Runs `start_drone2.py` which manages its DroneOS (from `DroneOS1` structure conceptually) and MAVSDK connection.
- **Drone 3:** Runs `start_drone3.py` which manages DroneOS2 and MAVSDK connection.

Commands from the Mobile App are sent to the Relay via WebSocket, and then distributed via UDP to the respective drones based on their drone IDs (`drone1`, `drone2`, `drone3`). Telemetry flows back via UDP from the drones to the Relay, and then via WebSocket to the Mobile App.

2. PROJECT DIRECTORY STRUCTURE

The complete `PhoneOS/` directory structure and its significance:

- **DroneOS/**: The main OS layer for Drone 1. Contains adapters, core logic (`FlightManager`, `CommandHandler`, etc.), configurations, sensors, and shared utilities.
- **DroneOS1/**: The OS layer configuration/code structure specifically for Drone 2 (via `start_drone2.py`).
- **DroneOS2/**: The OS layer specifically modified or scoped for Drone 3 (via `start_drone3.py`). This allows isolated execution environments and configurations for multiple drones on shared infrastructure or parallel development branches.
- **relay/**: Contains `relay.py`. Responsible for bridging WebSocket connections from the mobile app to UDP packets for the drone swarm.
- **mobile/**: A Vite + React + Capacitor web/mobile application. Used as the Ground Control Station.
- **missions/**: Stores automated flight plans and mission files.
- **tests/**: Automated test suite for validating MAVSDK and DroneOS functionality.
- **logs/**: Telemetry and system logs generated by DroneOS execution.

- **deploy/**: Systemd services (`droneos.service`) and deployment scripts.
- **venv/**: Python Virtual Environment containing installed dependencies (MAVSDK, grpcio, etc.).
- **start_drone2.py / start_drone3.py**: Entry points for starting the Drone 2 and Drone 3 environments, ensuring the correct MAVSDK instance and DroneOS namespaces are used.
- **deploy.sh / setup_pi.sh / setup_android.sh**: Helper scripts for bootstrapping environments.
- **test_mavsdk.py**: A diagnostic script to verify PX4 connection before running full DroneOS.

3. DroneOS / DroneOS1 / DroneOS2

Difference and Role

- **DroneOS**: Used by Drone 1. Represents the standard core architecture.
- **DroneOS1**: Used by Drone 2.
- **DroneOS2**: Used by Drone 3. The separation allows each drone to have dedicated adapter namespaces, configurations (`configs/drone.yaml`), and logic adjustments while sharing the same underlying project directory when tested locally, or distinct branches when deployed.

Component Architecture

- **Adapter Layer & Factory**: `adapters/factory.py` selects the correct connection adapter (e.g., `px4_adapter.py`) to interface with MAVSDK.
- **FlightManager**: Handles high-level flight state, arming, takeoff, and landing routines.
- **CommandHandler**: Ingests commands from the Relay (e.g., "ARM drone2") and routes them to FlightManager.
- **Safety**: Evaluates safety parameters (e.g., battery levels, GPS lock) before allowing operations.
- **TelemetryPublisher**: Publishes drone telemetry back to the Relay via UDP.
- **HealthMonitor / BatteryMonitor**: Checks system health, hardware status, and battery percentage.
- **SwarmMembership**: Manages the drone's status within the multi-drone network.

- **MissionManager / NavigationManager:** Handles autonomous waypoint missions and spatial navigation.
- **CollisionAvoidance / DecisionEngine:** Advanced safety mechanisms to prevent physical collisions in swarm mode.

4. RASPBERRY PI PREPARATION

[ALL DRONES - Raspberry Pi]

Requirements

- OS: Raspberry Pi OS Lite (64-bit recommended)
- Network: Static/reserved IP highly recommended for stable UDP peer discovery.

Hostname & Setup

Set unique hostnames: `drone1`, `drone2`, `drone3`.

```
sudo raspi-config # System Options -> Hostname
```

Git & Python Installation

```
sudo apt update sudo apt install git python3 python3-venv python3-pip
```

UART Configuration

You must disable the serial console but enable the serial port to allow Pixhawk connection.

In `/boot/firmware/config.txt`, ensure:

```
enable_uart=1
```

In `/boot/firmware/cmdline.txt`, ensure `console=serial0,115200` is **REMOVED**.

Add the user to the dialout group:

```
sudo usermod -a -G dialout $USER sudo reboot
```

Verify UART

After reboot:

```
ls -l /dev/serial0 readlink -f /dev/serial0
```

Expected Output: It should point to /dev/ttyAMA0 or /dev/ttyS0 depending on the Pi model.

5. PIXHAWK / PX4 CONNECTION

```
Raspberry Pi | | UART (/dev/serial0) v Pixhawk / PX4 (TELEM1 or TELEM2)
```

- **Baud Rate:** 115200
- **Connection String:** serial:///dev/serial0:115200

Verifying MAVSDK Server Connectivity

[DRONE - Raspberry Pi]

```
MAVSDK_SERVER="$PWD/venv/lib/python3.13/site-packages/mavsdk/bin/mavsdk_server"  
"$MAVSDK_SERVER" -p 50051 serial:///dev/serial0:115200
```

Expected Output:

```
Waiting to discover system... Component Autopilot added System discovered  
Server started Server set to listen on 0.0.0.0:50051
```

6. PYTHON VIRTUAL ENVIRONMENT

[ALL MACHINES]

```
cd ~/Projects/PhoneOS python3 -m venv venv source venv/bin/activate
```

Verify activation: `which python` should point to `venv/bin/python`.

Install Dependencies:

```
pip install -r DroneOS/requirements.txt
```

Required packages include: mavsdk, PyYAML, psutil, pydantic, grpcio, protobuf, typing-extensions, websockets.

(For offline installation, use `pip download` on a connected machine, transfer the files, and run `pip install --no-index --find-links=./dir -r requirements.txt`)

7. DRONE 1 SETUP

[DRONE 1 - Raspberry Pi]

This starts the main DroneOS instance for Drone 1.

```
cd ~/Projects/PhoneOS source venv/bin/activate sudo systemctl stop
swarmos-drone.service 2>/dev/null || true sudo systemctl stop
swarmos-phone-relay.service 2>/dev/null || true sudo pkill -9 -f
'DroneOS.main|DroneOS1.main|mavsdk_server' || true sleep 2 PYTHONUNBUFFERED=1
python -m DroneOS.main
```

Expected Logs:

```
Starting DroneOS Node: drone1 PX4 CONNECTING PX4 CONNECTED UdpNetworkAdapter
started TelemetryPublisher loops started DroneOS is running
```

8. DRONE 1 RELAY

[DRONE 1 / PC / SERVER]

The Relay bridges the Mobile App (WebSocket) to the Swarm (UDP).

```
cd ~/Projects/PhoneOS source venv/bin/activate PYTHONUNBUFFERED=1 python
relay/relay.py
```

It listens for WebSocket connections on Port 8080 (or as configured) and routes UDP traffic to peer endpoints.

9. DRONE 2 SETUP

[DRONE 2 - Raspberry Pi]

Drone 2 uses a managed script that orchestrates the MAVSDK server and DroneOS instance.

```
cd ~/Projects/PhoneOS source venv/bin/activate PYTHONUNBUFFERED=1 python
start_drone2.py
```

`start_drone2.py` ensures the correct MAVSDK gRPC endpoint and serial connection are established. **Important:** DroneOS must NOT kill the managed MAVSDK server. If you see "Killing orphaned mavsdk_server", it means the server crashed or was terminated, resulting in `Stream removed (Socket closed)` or `TimeoutError`.

10. DRONE 3 SETUP

[DRONE 3 - Raspberry Pi]

Drone 3 uses DroneOS2.

```
sudo pkill -9 -f mavsdk_server 2>/dev/null || true sudo pkill -9 -f
'start_drone3.py|DroneOS2.main|relay/relay.py' 2>/dev/null || true sleep 2 cd
~/Projects/PhoneOS source venv/bin/activate PYTHONUNBUFFERED=1 python
start_drone3.py
```

Using `venv/lib/python3.13/site-packages/mavsdk/bin/mavsdk_server` directly ensures the correct binary is executed, as `get_mavsdk_server_filepath` might fail with some versions.

11. MULTI-DRONE STARTUP

Correct Startup Order: 1. **Drone 1 Terminal 1:** Relay (`python relay/relay.py`) 2. **Drone 1 Terminal 2:** DroneOS 1 (`python -m DroneOS.main`) 3. **Drone 2 Terminal:** Drone 2 (`python start_drone2.py`) 4. **Drone 3 Terminal:** Drone 3 (`python start_drone3.py`)

Each Raspberry Pi runs independently. The Mobile Ground Station communicates via the Relay, which broadcasts and listens to UDP ports. Each drone identifies its target commands via its drone ID (`drone1`, `drone2`, `drone3`).

12. HEALTH CHECK

To verify the system is running:

Check Processes:

```
pgrep -af 'DroneOS|mavsdk_server|relay'
```

Check Ports:

```
ss -ltnp | grep -E ':50051|:14550|:14551|:8080'
```

Check UART:

```
ls -l /dev/serial0 readlink -f /dev/serial0
```

Check Packages:

```
python --version pip list
```

13. MAVSDK CONNECTION TEST

Run `test_mavsdk.py` to isolate PX4 issues from DroneOS issues.

```
python test_mavsdk.py
```

This tests the gRPC connection, MAVSDK server discovery, and PX4 connection. *Expected Output:*

```
CONNECTED PX4 CONNECTION = OK
```

Note: "MAVSDK server reachable" simply means the python grpc client talked to the binary server. "Pixhawk actually connected" means the UART link to PX4 was successfully established.

14. PX4 ARM / PREFLIGHT TROUBLESHOOTING

Rule: PX4 CONNECTED != ARMABLE

The telemetry reports health fields: gyro, accel, mag, local, global, home, armable.

If you receive **PreArm: Need Position Estimate**, it means the drone lacks GPS lock. Indoor testing will often connect successfully but refuse to ARM due to this PX4 safety condition. This is not a PhoneOS failure. Outdoor testing or disabling GPS requirements in PX4 parameters is required for arming.

15. MOBILE APP

The Mobile App serves as the Ground Control Station. - **Framework:** Vite + React + Capacitor (as defined in package.json). - **Networking:** Connects to Relay via WebSocket. - **Features:** Telemetry display, command buttons (ARM, TAKEOFF), drone selection (drone1, drone2, etc.).

Directory Structure

- src/: React components.
- package.json: Build configurations and dependencies.
- vite.config.js: Vite build system configuration.

16. MOBILE APP BUILD

[PC/LAPTOP]

Install Dependencies:

```
cd ~/Projects/PhoneOS/mobile npm install
```

Development Mode:

```
npm run dev
```

Build for Web/Production:

```
npm run build
```

Generate APK (Android):

```
npm run android:debug
```

This command runs `npx cap sync android` and invokes `gradle` to assemble the debug APK. Find it in `android/app/build/outputs/apk/debug/`.

Configuration: Modify WebSocket IP/Port in the React `src/` networking logic to point to the Relay's IP. Rebuild after changes.

17. MOBILE APP UPDATE WORKFLOW

1. Modify code in `src/`.
2. Test locally using `npm run dev`.
3. Build using `npm run build`.
4. Install APK to Android device.
5. Test network and drone commands.
6. Commit changes:

```
git status git add -A git commit -m "Update mobile UI and commands" git push  
origin main
```

`git add -A` explicitly tracks all new, modified, and deleted files.

18. GITHUB REPOSITORY MANAGEMENT

[ALL MACHINES]

To pull the latest updates across all Pis:

```
git pull origin main
```

Use `git status` to view modifications. **WARNING:** Do NOT use `git clean -fd` unless you intentionally want to permanently delete all untracked files (including local logs, test scripts, etc.).

19. CONFIGURATION GUIDE

Configurations are stored in `configs/` subdirectories.

- **drone.yaml**: Sets `drone_id` (e.g., `drone1`), vehicle name, UART paths.
- **network.yaml**: Sets UDP ports, WebSocket ports, and relay configurations.
- **flight.yaml**: Flight limits, altitude constraints.

Ensure that `drone1`, `drone2`, and `drone3` each have their respective configuration files correctly pointing to their unique IDs to prevent duplicate command processing.

20. NETWORKING

Topology:

```
Laptop / Mobile | Wi-Fi | Network Router / | Pi1 Pi2 Pi3 D1 D2 D3 | | | PX4
PX4 PX4
```

- **UDP 14550/14551**: Used for Swarm peer communication.
- **WebSocket 8080**: Mobile App to Relay communication.
- **MAVSDK gRPC 50051**: Local Python to MAVSDK binary communication.

Peer Discovery: Look for `Connection established! Learned peer endpoint:`
`x.x.x.x:14551` in the logs to verify UDP discovery.

21. COMMON ERRORS

SYMPTOM	CAUSE	CHECK	FIX	EXPECTED RESULT
---------	-------	-------	-----	-----------------

ModuleNotFoundError: No module named pydantic/yaml	Missing venv dependency	<code>pip list</code>	<code>pip install -r requirements.txt</code>	Package installed
MAVSDK server port 50051 already in use	Ghost process running	<code>pgrep -af mavsdk</code>	<code>sudo pkill -9 -f mavsdk</code>	Port freed
Stream removed (Socket closed)	mavsdk_server crashed/killed	<code>journalctl</code>	Ensure DroneOS doesn't kill managed server	Reconnects
PX4 CONNECTED but ARM FAILED	Pre-arm conditions	Telemetry logs	Take outdoors for GPS lock	Arm succeeds
UART device missing	Pi config incorrect	<code>ls -l /dev/serial0</code>	Fix <code>config.txt</code>	<code>serial0</code> exists
Serial console conflict	Console blocking UART	<code>cmdline.txt</code>	Remove <code>console=serial0</code>	Clean UART <code>comms</code>
Drone ID mismatch	Config error	<code>drone.yaml</code>	Fix <code>drone_id</code>	Commands route correctly
Duplicate ARM command	Multiple IDs match	Network logs	Isolate IDs per Pi	Single drone arms

22. SAFE CLEANUP COMMANDS

[ALL MACHINES]

Use these commands ONLY before starting a fresh instance to clear zombie processes:

```
sudo pkill -9 -f mavsdk_server sudo pkill -9 -f DroneOS sudo pkill -9 -f relay/relay.py
```

Blindly killing processes during active flight can result in drone crash or loss of control.

23. SYSTEMD SERVICES

Services: `swarmos-drone.service`, `swarmos-phone-relay.service`

Commands: `- systemctl status <service>` `- systemctl start <service>` `- systemctl stop <service>` `- systemctl restart <service>` `- systemctl enable <service>` (start on boot)

For debugging, it is highly recommended to stop the services and run Python manually to see live logs. Inspect background logs via: `journalctl -u swarmos-drone.service -f`.

24. LOGGING

Log prefixes identify the failing layer: - **PX4Adapter:** Hardware/MAVSDK connection. -

DroneOS_Main / AdapterFactory: Core initialization. - **CommandHandler / FlightManager:**

Command routing and state changes. - **PhoneOS_Relay / UdpNetworkNode:** Networking and bridging.

25. COMMAND FLOW

Example: Mobile sends ARM drone2

1. **Mobile App** sends WebSocket JSON.
2. **Relay** receives WebSocket, broadcasts UDP packet.
3. **Drone2 UdpNetworkNode** receives UDP packet.
4. **CommandHandler** verifies `target == drone2`.
5. **FlightManager** initiates arming sequence.

6. **PX4 Adapter** calls MAVSDK action.
7. **MAVSDK Server** translates to MAVLink over UART.
8. **Pixhawk (PX4)** physically arms motors.

Telemetry flows in the exact reverse order.

26. SWARM OPERATION

Multi-drone swarm control separates logic based on drone identity. Simultaneous startup requires independent Raspberry Pis. Commands are targeted (e.g. `ARM drone1`), while telemetry is tagged with the drone ID. Swarm manager handles collision avoidance and formation control, distinguishing multi-drone network state from single-drone manual control.

27. DEVELOPMENT GUIDE

- **Safe to Modify:** `CommandHandler`, `FlightManager`, Mobile UI, Configuration files.
- **Treat as Infrastructure (Modify with Caution):** `PX4 Adapter` (MAVSDK grpc calls), Relay UDP core, `UdpNetworkNode`.

28. ADDING A NEW DRONE (Drone 4)

1. Flash Raspberry Pi OS.
2. Setup Network and SSH.
3. Clone PhoneOS.
4. Configure UART and connect Pixhawk.
5. Create `configs/drone.yaml` with `drone_id: drone4`.
6. Create `start_drone4.py` mapping to a new isolated namespace or reused structure.
7. Update Mobile UI to add a `drone4` button.
8. Test and Git Commit.

29. COMPLETE FRESH INSTALLATION (START FROM ZERO)

1. **Step 1:** Raspberry Pi OS Installation
2. **Step 2:** Network Configuration (Static IP)
3. **Step 3:** SSH Access Setup
4. **Step 4:** UART Enable (config.txt, remove cmdline.txt console)
5. **Step 5:** Git Clone PhoneOS repository
6. **Step 6:** Python venv creation
7. **Step 7:** Install dependencies (pip install -r)
8. **Step 8:** Configure drone_id and ports
9. **Step 9:** Connect Pixhawk to Pi GPIO 14/15
10. **Step 10:** Verify MAVSDK using test_mavsdk.py
11. **Step 11:** Start Relay
12. **Step 12:** Start DroneOS
13. **Step 13:** Build Mobile App and install APK
14. **Step 14:** Test Network & Telemetry
15. **Step 15:** Test ARM command (Ensure GPS lock)
16. **Step 16:** Outdoor flight test

30. QUICK START

Drone 1 (Main OS + Relay):

```
cd ~/Projects/PhoneOS source venv/bin/activate sudo systemctl stop
swarmos-drone.service 2>/dev/null || true sudo systemctl stop
swarmos-phone-relay.service 2>/dev/null || true sudo kill -9 -f
'DroneOS.main|DroneOS1.main|mavsdk_server' || true sleep 2 PYTHONUNBUFFERED=1
python -m DroneOS.main # In another terminal on Drone 1 PYTHONUNBUFFERED=1
```

```
python relay/relay.py
```

Drone 2:

```
cd ~/Projects/PhoneOS source venv/bin/activate PYTHONUNBUFFERED=1 python  
start_drone2.py
```

Drone 3:

```
sudo pkill -9 -f mavsdk_server 2>/dev/null || true sudo pkill -9 -f  
'start_drone3.py|DroneOS2.main|relay/relay.py' 2>/dev/null || true sleep 2 cd  
~/Projects/PhoneOS source venv/bin/activate PYTHONUNBUFFERED=1 python  
start_drone3.py
```

31. FINAL VALIDATION CHECKLIST

- ☐ Raspberry Pi boots
- ☐ SSH works
- ☐ Network works
- ☐ UART enabled
- ☐ `/dev/serial0` exists
- ☐ Pixhawk detected
- ☐ Python venv works
- ☐ Dependencies installed
- ☐ MAVSDK installed
- ☐ MAVSDK server works
- ☐ PX4 connected
- ☐ Relay running
- ☐ UDP communication works
- ☐ WebSocket works
- ☐ Mobile app builds
- ☐ Mobile app connects

- ☐ Telemetry received
- ☐ Drone IDs correct
- ☐ Drone 1 works
- ☐ Drone 2 works
- ☐ Drone 3 works
- ☐ Multi-drone communication works
- ☐ ARM preflight checks pass
- ☐ Outdoor flight test completed